Trabajo Práctico – Motor de búsqueda de funciones similares vía AST

1. Decisiones de diseño

Para desarrollar el trabajo decidimos utilizar Python y su módulo ast, que nos permite analizar el código y obtener la estructura de las funciones sin necesidad de crear un analizador sintáctico desde cero.

Organizamos el programa en cuatro archivos principales, separando las tareas de extracción, normalización, comparación y búsqueda. Elegimos hacerlo de esta manera para mantener el código ordenado y que cada parte tenga una función específica.

Primero, el programa recorre la carpeta indicada y busca todos los archivos Python, incluyendo los que están dentro de subcarpetas. De cada función encontrada guarda su nombre, el archivo al que pertenece, el número de línea y su árbol sintáctico.

Después normalizamos los árboles para que los nombres utilizados en el código no afecten la comparación. Para esto, reemplazamos las variables y parámetros por nombres genéricos como VAR1, VAR2, etc., manteniendo la correspondencia cuando una misma variable aparece varias veces. También reemplazamos los valores numéricos y textos por LIT, pero conservamos las operaciones y estructuras del código.

Una vez normalizados, recorremos los árboles en preorden para generar una secuencia de tokens que represente su estructura.

Por último, el usuario puede seleccionar una función desde la consola y el programa devuelve las funciones más similares, ordenadas según su puntuación.

2. Métrica de similitud elegida

Para comparar las funciones elegimos utilizar la similitud de Jaccard sobre trigramas, es decir, grupos de tres tokens consecutivos. El cálculo consiste en comparar los grupos de tokens de ambas funciones y determinar qué proporción tienen en común respecto del total de grupos diferentes. El resultado es un valor entre 0 y 1, donde 1 representa la mayor similitud posible.

Elegimos esta métrica porque es relativamente sencilla de implementar y permite encontrar coincidencias entre partes de las funciones sin exigir que toda la secuencia sea idéntica. Decidimos utilizar grupos de tres tokens porque nos permiten tener en cuenta el orden de los elementos cercanos, sin trabajar con fragmentos demasiado grandes.

También establecimos un umbral de 0,70 (70 %). Cuando dos funciones alcanzan o superan ese valor, el programa las marca como posibles duplicados. Esto no significa que sean necesariamente iguales, sino que tienen una similitud estructural que consideramos suficiente como para revisarlas.

3. Limitaciones conocidas

Una de las principales limitaciones es que nuestro programa compara la estructura de las funciones, pero no comprueba si realmente hacen lo mismo.

Por ejemplo, dos funciones que obtienen el mismo resultado utilizando instrucciones diferentes pueden recibir una similitud baja. Esto puede suceder si una utiliza un for y otra un while, o si resuelven el mismo problema de maneras distintas.

También puede disminuir la similitud cuando se cambia el orden de varias instrucciones o se agregan pasos intermedios, aunque las funciones sigan siendo parecidas. Por otro lado, al reemplazar todos los valores numéricos y textos por LIT, dos funciones que trabajan con valores diferentes pueden terminar teniendo una estructura normalizada idéntica. Otra limitación es que Jaccard utiliza conjuntos de trigramas, por lo que no tiene en cuenta cuántas veces se repite cada uno.

Por estas razones, los resultados sirven para encontrar posibles coincidencias, pero no garantizan que dos funciones sean equivalentes.

4. Posibles mejoras

Una mejora que podríamos incorporar sería probar otros tamaños de n-gramas y comparar sus resultados, para determinar cuál permite reconocer mejor las funciones similares.

También podríamos implementar otra métrica, como la distancia de Levenshtein, y comparar su funcionamiento con Jaccard. Otra posibilidad sería mejorar el reconocimiento de funciones que tienen instrucciones reordenadas o que realizan la misma tarea utilizando estructuras diferentes.

Por ahora, decidimos mantener una implementación sencilla, que cumple con el objetivo del trabajo y permite comprender fácilmente cómo se realiza cada etapa de la comparación.